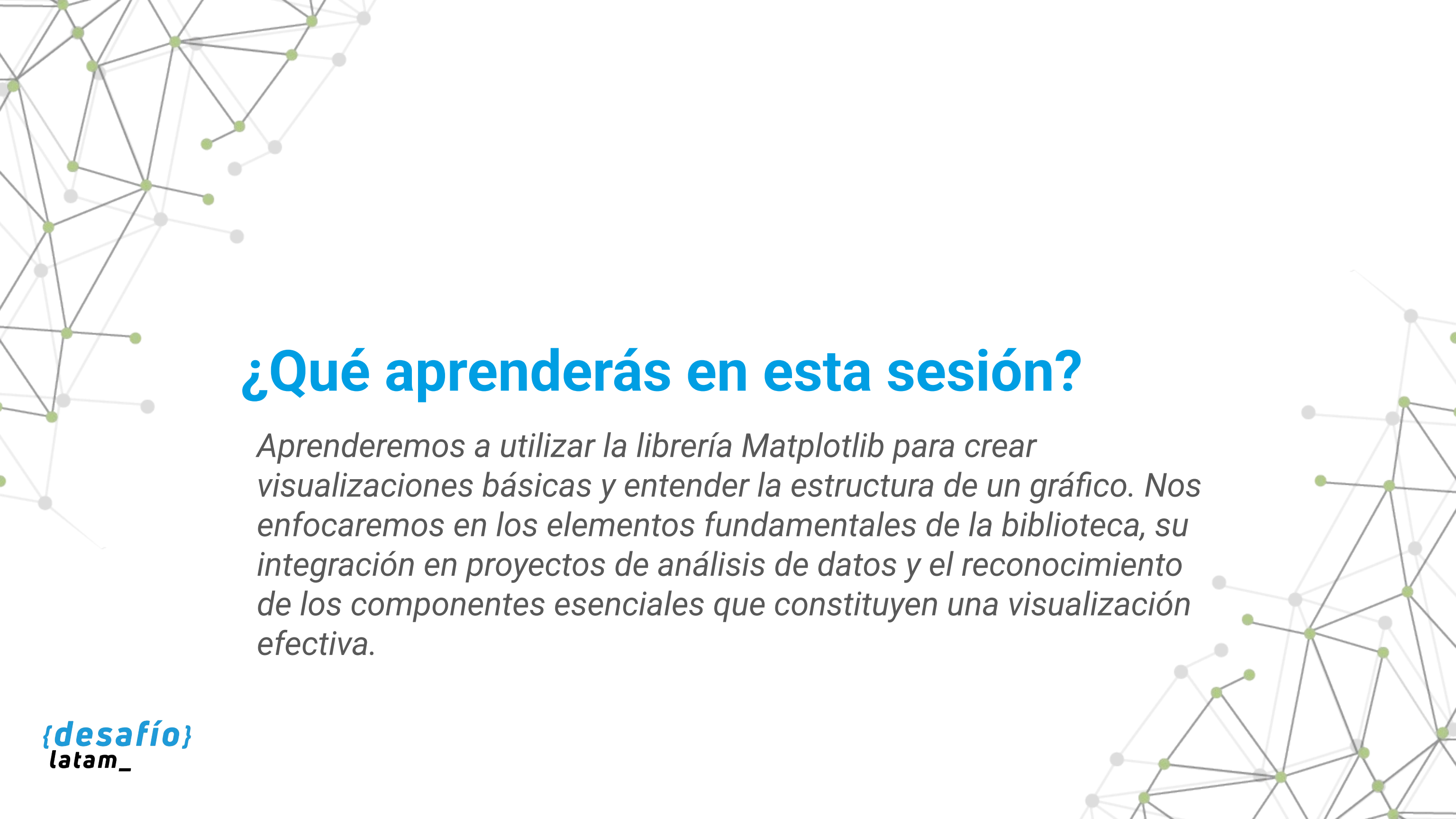




Visualización de datos

Clase 5 - Introducción a la visualización con Matplotlib



¿Qué aprenderás en esta sesión?

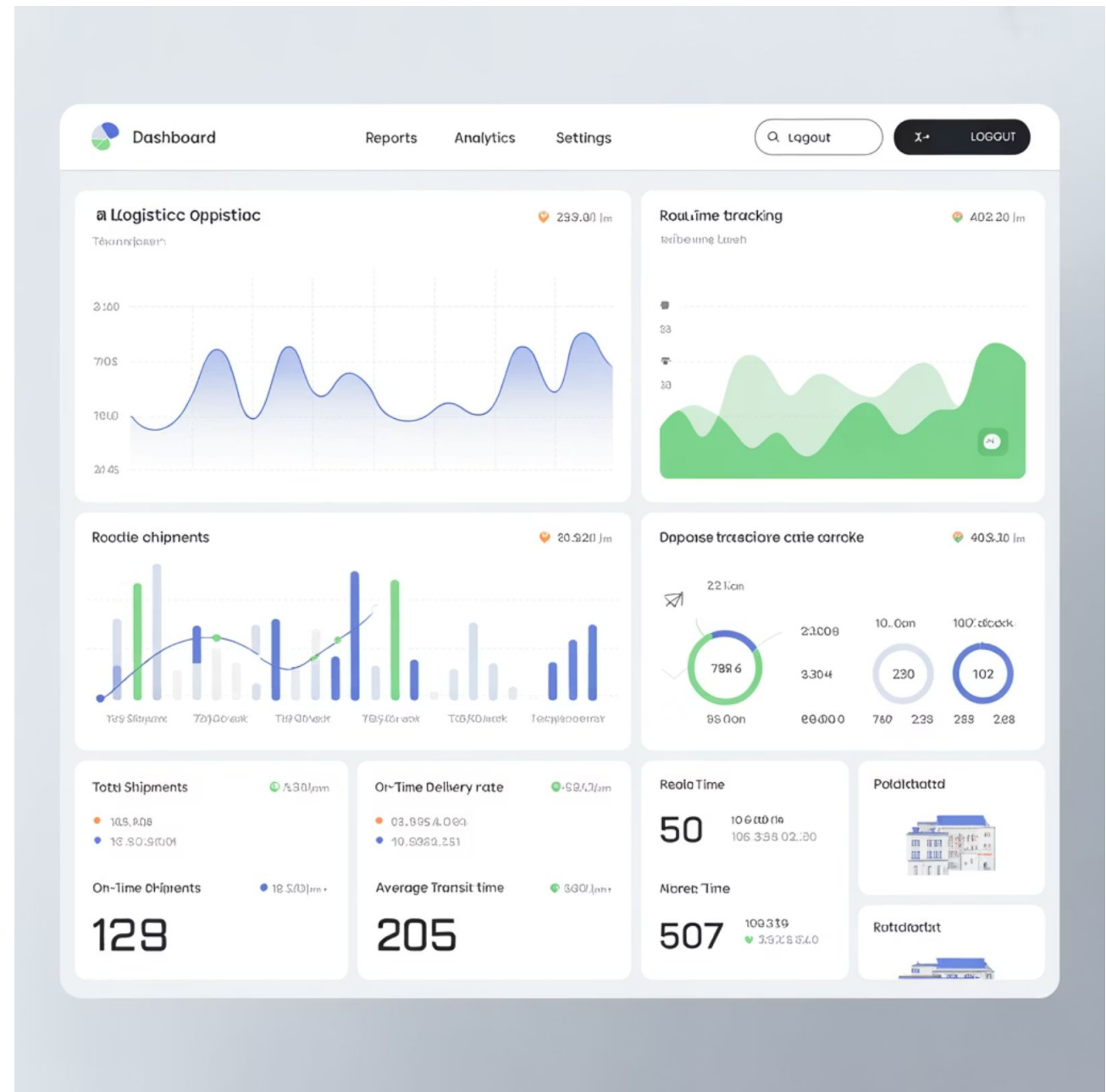
Aprenderemos a utilizar la librería Matplotlib para crear visualizaciones básicas y entender la estructura de un gráfico. Nos enfocaremos en los elementos fundamentales de la biblioteca, su integración en proyectos de análisis de datos y el reconocimiento de los componentes esenciales que constituyen una visualización efectiva.

Desafío inicial

Imagina que formas parte del equipo de análisis de una empresa de logística. Se te pide presentar visualmente la evolución semanal de entregas exitosas por zona geográfica. Sin embargo, solo cuentas con los datos en bruto y necesitas una herramienta flexible para graficar.

¿Cómo puedes generar gráficos personalizados que te permitan construir presentaciones claras para un público no técnico?

Para resolverlo deberás comprender las capacidades de Matplotlib como librería de visualización además de aprender a importar y utilizar la herramienta correctamente. De igual forma, debes poder identificar los componentes esenciales que conforman una figura (ejes, títulos, leyendas, etc.) para ajustar los gráficos según las necesidades de comunicación del equipo.



`/* Librería Matplotlib */`

Características de la librería Matplotlib

¿Por qué se considera Matplotlib una herramienta fundamental en el ecosistema de visualización en Python?

Matplotlib es una de las bibliotecas más robustas y versátiles del ecosistema

Python para la generación de gráficos estáticos, animados e interactivos. Fue

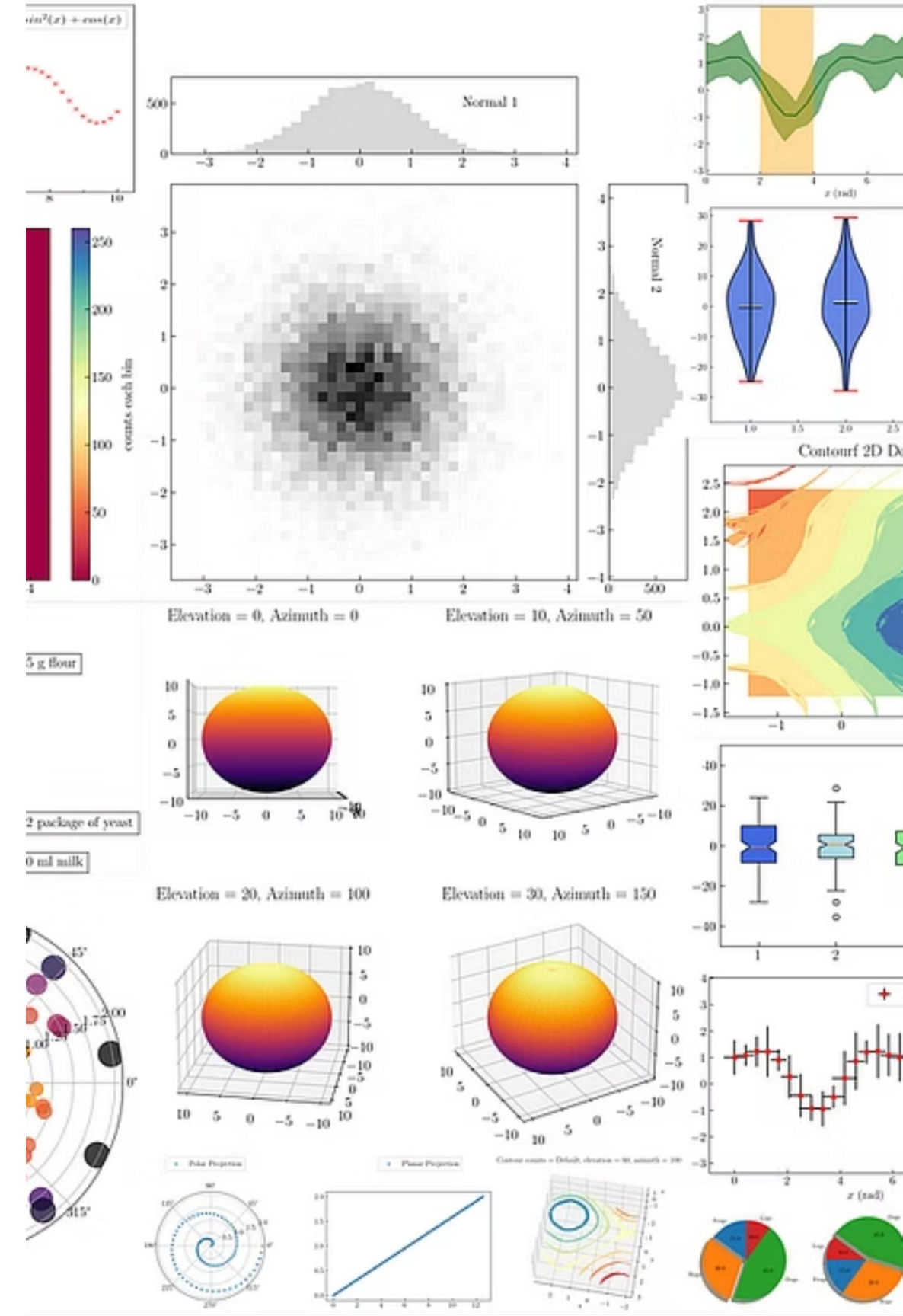
creada en 2003 por John D. Hunter y se ha convertido en el estándar de facto

para visualización en entornos científicos y productivos.

A diferencia de bibliotecas como Seaborn (que está construida sobre Matplotlib),

esta ofrece control total sobre cada elemento visual del gráfico, lo cual es ideal

para situaciones donde se necesita alta personalización.



Principales características de Matplotlib

- Permite crear gráficos 2D de líneas, barras, histogramas, dispersión, áreas, tortas, etc.
- Admite salidas en múltiples formatos: PNG, PDF, SVG, EPS, entre otros.
- Compatible con interfaces gráficas como Qt, GTK y Tkinter.
- Tiene una arquitectura basada en objetos, permitiendo el control detallado del gráfico.
- Integra bien con bibliotecas como NumPy, pandas y Seaborn.

Ejemplo:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]
plt.plot(x, y)
plt.title("Ventas semanales")
plt.xlabel("Semana")
plt.ylabel("Unidades vendidas")
plt.show()
```

Buenas prácticas al usar Matplotlib

Antes de crear visualizaciones con Matplotlib, es fundamental conocer ciertas recomendaciones que aseguren resultados claros, eficientes y visualmente consistentes. Estas buenas prácticas no solo mejoran la presentación, sino también optimizan el rendimiento del código y su integración en entornos productivos. A continuación, destacamos algunas estrategias esenciales para sacar el mayor provecho de esta librería:



Control total

Utiliza Matplotlib para tener control total sobre detalles gráficos.



Integración con pandas

Combínala con pandas para graficar directamente DataFrames.



Coherencia visual

Usa temas predefinidos (`plt.style.use()`) para mantener coherencia visual.



Reutilización

Guarda tus gráficos con `plt.savefig()` para reutilización.

Errores comunes

El uso inadecuado de Matplotlib puede llevar a visualizaciones confusas, poco eficientes o incluso incorrectas. Identificar los errores más frecuentes permite evitarlos y desarrollar visualizaciones más profesionales desde el inicio. A continuación, presentamos los fallos más habituales que debes evitar al trabajar con esta herramienta:

Omitir `plt.show()`

Omitir `plt.show()` en entornos sin renderizado automático.

No cerrar figuras

No cerrar figuras anteriores con `plt.close()` cuando se generan muchos gráficos.

Confundir interfaces

Confundir interfaces de estilo procedural y orientada a objetos.

Matplotlib es una herramienta clave para visualizar datos con precisión y flexibilidad. Ahora que conocemos sus capacidades generales, veamos cómo integrarla correctamente en un proyecto.

Importación de la librería

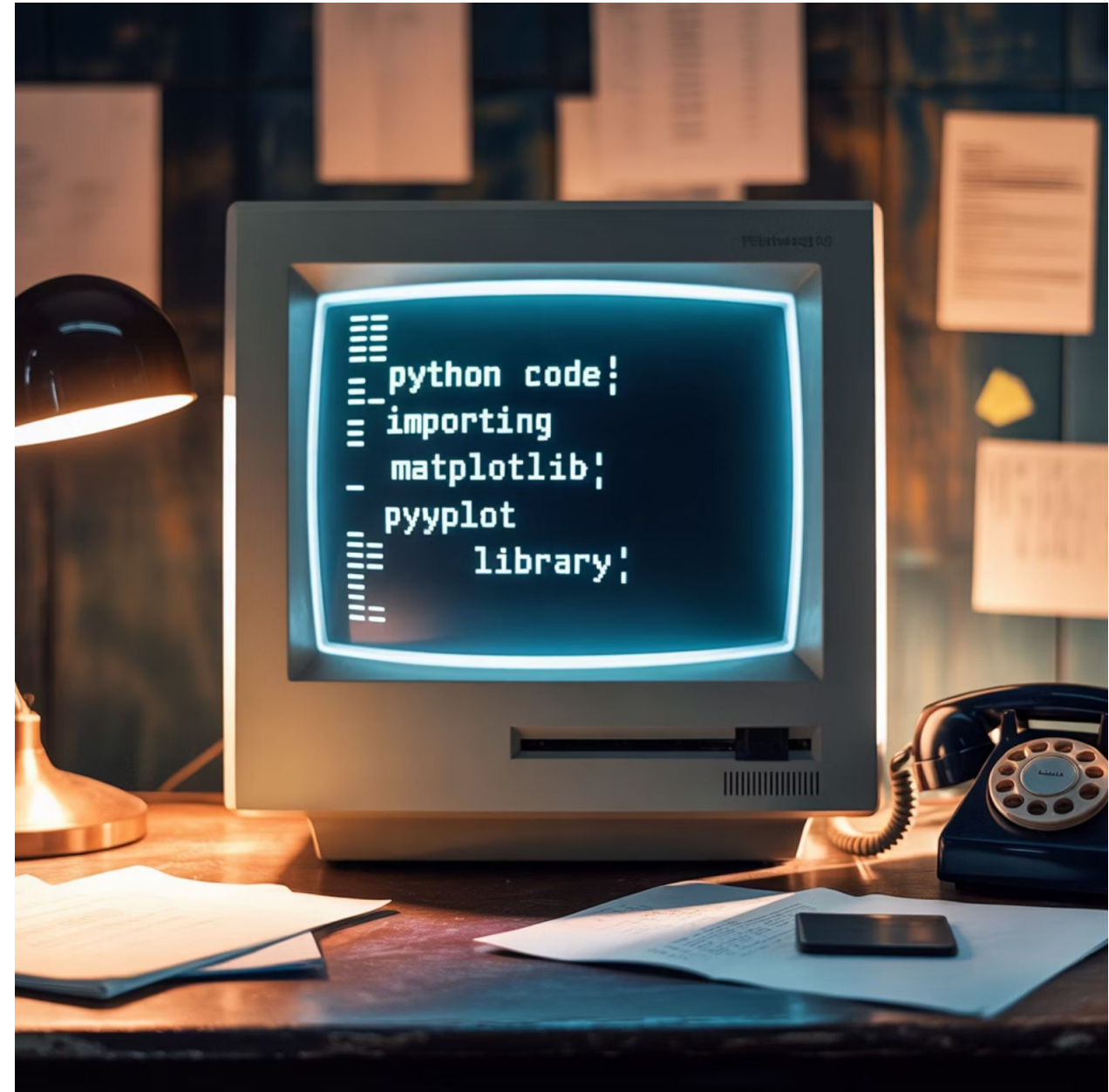
¿Cuál es la forma recomendada y estándar de importar Matplotlib para empezar a trabajar?

Matplotlib puede ser utilizada mediante dos interfaces: una basada en comandos similares a MATLAB (procedural) y otra orientada a objetos (OOP). Aunque ambas son funcionales, la interfaz procedural es más común para tareas rápidas y educativas.

La sublibrería más usada es pyplot, que se importa de la siguiente manera:

```
import matplotlib.pyplot as plt
```

Esto permite usar funciones como `plt.plot()`, `plt.title()`, `plt.xlabel()` y otras.

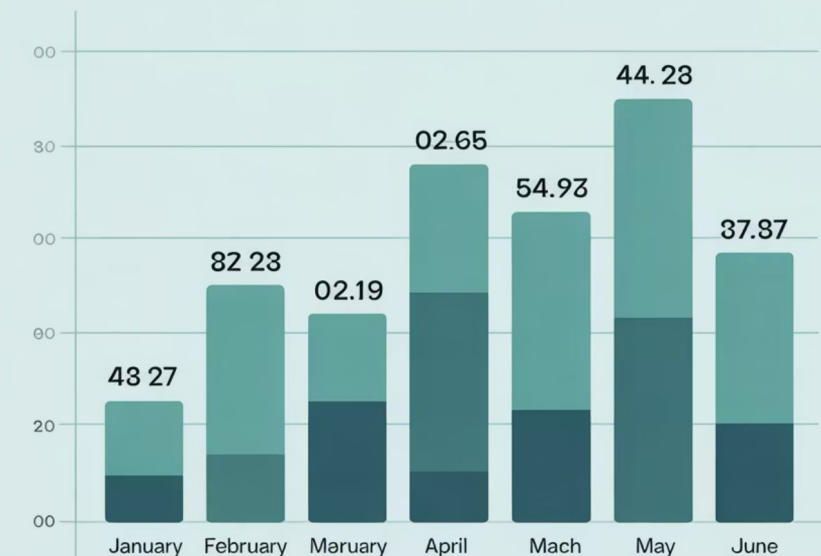


Ejemplo práctico

Uno de los primeros pasos para familiarizarnos con pyplot, el módulo más utilizado de Matplotlib, es entender cómo generar un gráfico de barras simples. Esto nos permite visualizar rápidamente cantidades asociadas a categorías discretas (como meses, productos, zonas, etc.).

El siguiente ejemplo muestra cómo graficar las ventas por mes utilizando funciones básicas de pyplot. A través de este código, se introducen elementos esenciales como títulos, etiquetas de ejes y cuadrículas para facilitar la lectura del gráfico:

```
import matplotlib.pyplot as plt
meses = ["Ene", "Feb", "Mar", "Abr"]
ventas = [100, 120, 90, 150]
plt.bar(meses, ventas)
plt.title("Ventas por mes")
plt.xlabel("Mes")
plt.ylabel("Monto en miles")
plt.grid(True)
plt.show()
```

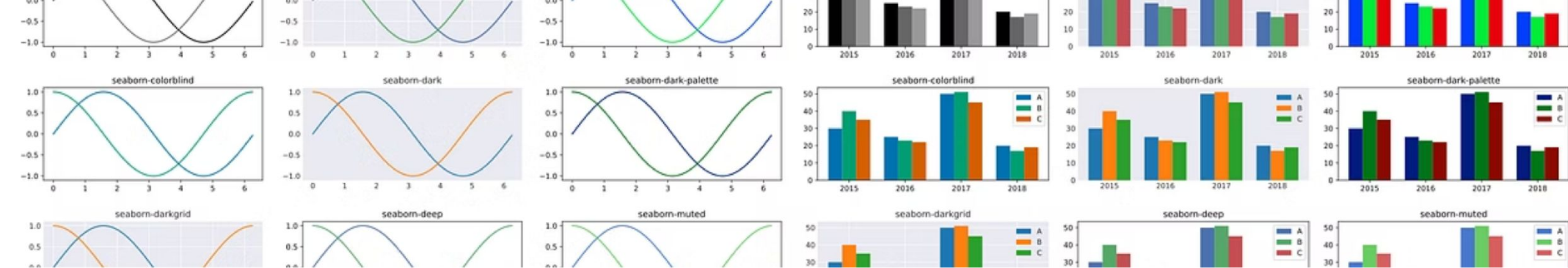


Este código ilustra cómo usar funciones de pyplot para construir un gráfico legible y personalizable. A continuación, se presenta una tabla con las funciones más comunes de pyplot, que te servirán como referencia rápida para crear distintos tipos de gráficos.



Comandos básicos de pyplot

Función	Tipo de gráfico	Aplicación típica
plot()	Línea	Seguimiento de tendencias en el tiempo (series temporales)
bar()	Barras verticales	Comparar cantidades por categoría
barh()	Barras horizontales	Útil para etiquetas largas o categorías numerosas
scatter()	Diagrama de dispersión	Visualizar relaciones entre variables numéricas
hist()	Histograma	Distribución de frecuencias
title()	Título del gráfico	Identificación del propósito del gráfico
xlabel(), ylabel()	Etiquetas de ejes	Contextualización del eje X y Y



Estas funciones son la base para crear visualizaciones eficaces. Saber cuándo usar cada una te permitirá comunicar tus hallazgos de forma precisa y profesional. A medida que avances, se podrán combinar varias de ellas para construir visualizaciones más sofisticadas.

Buenas prácticas

Antes de comenzar a trabajar con Matplotlib, es importante considerar algunas prácticas que facilitarán tu trabajo, mejorarán la legibilidad del código y evitarán errores comunes al momento de crear visualizaciones. Estas buenas prácticas te ayudarán a mantener tu código ordenado, entendible y más fácil de depurar en proyectos colaborativos o personales.

Usa siempre un alias (plt)

Al importar matplotlib.pyplot, se recomienda utilizar el alias plt, ya que es el estándar en la comunidad y mejora la legibilidad y consistencia del código en diferentes fuentes y tutoriales.

Instala la librería si no está presente

Asegúrate de tener instalada la librería con `pip install matplotlib`. Esto evitará errores al momento de importar y trabajar con visualizaciones.

Ejecuta `plt.show()` para visualizar correctamente

Si trabajas fuera de entornos como Jupyter Notebook (por ejemplo, en scripts `.py`), es fundamental llamar explícitamente a `plt.show()` para que el gráfico se muestre.

Trabaja en entornos interactivos tipo Jupyter o IPython

Estas plataformas permiten una visualización automática de los gráficos y una edición más ágil del código, lo cual es especialmente útil durante la fase exploratoria de los datos.

Errores comunes

Muchas personas que comienzan a trabajar con Matplotlib cometen errores que pueden parecer pequeños, pero que afectan directamente la visualización o el flujo del análisis. A continuación se presentan algunos de los errores más habituales para que puedas evitarlos desde el principio:

Confundir funciones como plt.bar() y plt.plot()

Aunque ambas generan gráficos, plt.bar() es para barras categóricas mientras que plt.plot() dibuja líneas continuas. Usarlas indistintamente puede conducir a visualizaciones incorrectas.

Intentar graficar con listas desalineadas

Un error frecuente es que las listas de valores de los ejes X e Y no tienen la misma longitud, lo que provoca un error al ejecutar plt.plot() u otras funciones similares. Verifica siempre que ambas listas coincidan en número de elementos.

Importar sin alias (as plt)

Aunque técnicamente es válido importar matplotlib.pyplot sin un alias, esto genera un código más extenso y menos legible. Además, rompe con la convención utilizada en casi toda la documentación oficial y tutoriales.

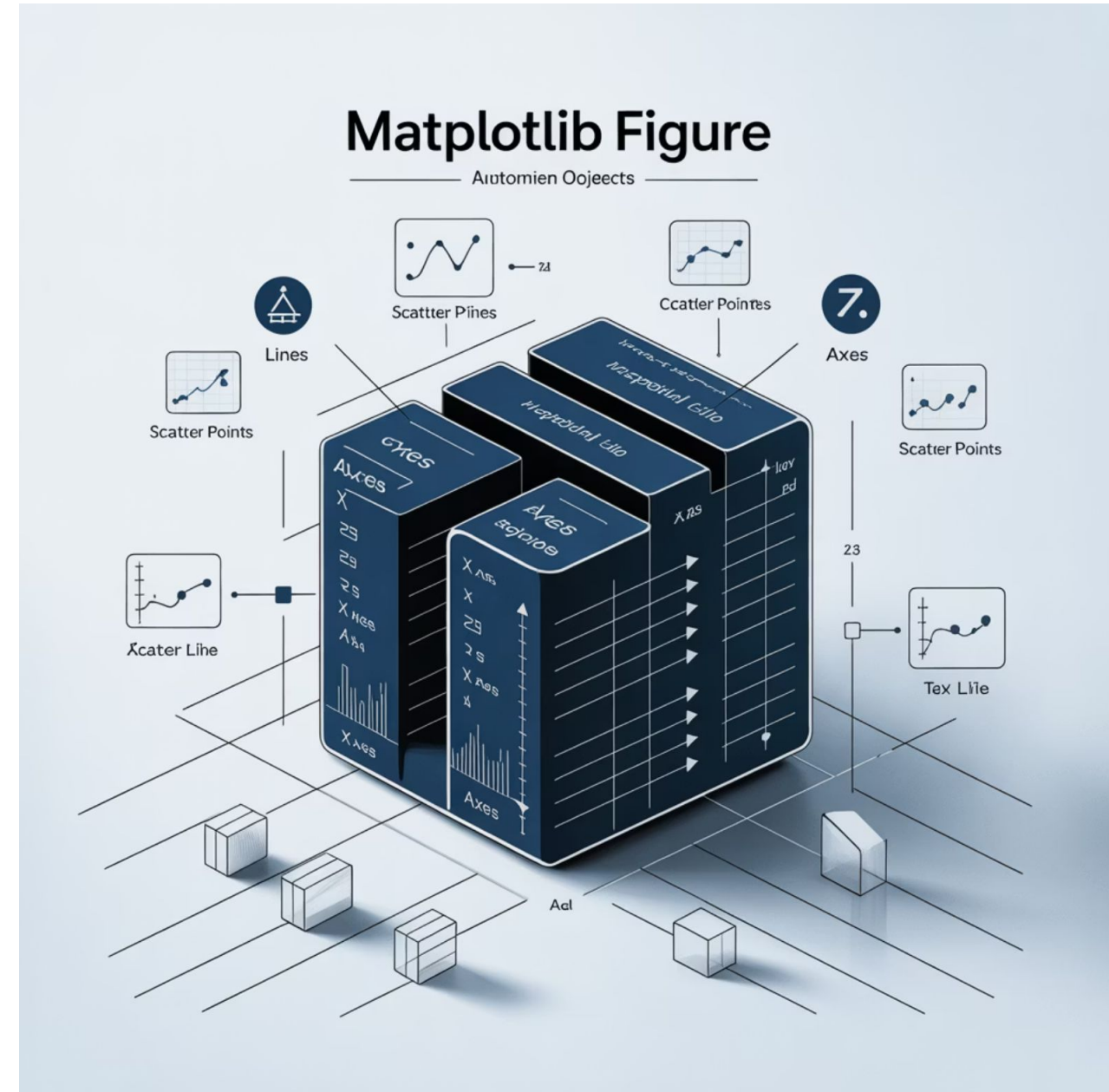
Componentes principales de un gráfico

¿Qué elementos estructurales hacen que un gráfico sea comprensible y profesional en entornos técnicos?

Un gráfico no es solo una representación visual de datos; es una herramienta de comunicación. En Matplotlib, cada visualización está compuesta por elementos organizados jerárquicamente, como la figura, los ejes y los objetos visuales que representan los datos. Entender esta estructura es clave para personalizar gráficos con precisión y adaptarlos a necesidades específicas del análisis profesional.

La mayoría de los gráficos de Matplotlib se construyen sobre dos clases principales:

- **Figure:** contenedor general de la visualización, puede incluir uno o más ejes.
- **Axes:** área específica donde se representa un conjunto de datos. No debe confundirse con los ejes X e Y.



Anatomía de un gráfico en Matplotlib

A continuación, se presenta un esquema conceptual que relaciona los elementos más importantes de un gráfico típico:

Elemento	Descripción
Figure	Objeto contenedor global. Puede incluir varios ejes (Axes).
Axes	Área de trazado donde se colocan los datos, etiquetas, título, etc.
Axis	Los ejes X o Y en sí mismos, contienen ticks y valores numéricos.
Tick	Marcadores en los ejes, indican valores individuales.
Label	Etiquetas asociadas a cada eje (xlabel, ylabel).
Title	Título general del gráfico o por cada Axes.
Legend	Explicación de lo que representa cada serie de datos.
Grid	Líneas auxiliares que mejoran la legibilidad numérica del gráfico.

Este desglose es importante porque cada uno de estos elementos puede controlarse de forma independiente, ofreciendo un grado alto de personalización.

Creación de una figura con subgráficos

El método recomendado para construir gráficos en Matplotlib, especialmente en contextos técnicos, es mediante la interfaz orientada a objetos:

```
import matplotlib.pyplot as plt
# Crear figura y eje
fig, ax = plt.subplots()
# Trazar línea
ax.plot([1, 2, 3, 4], [10, 20, 25, 30], label="Ventas")
# Personalización del gráfico
ax.set_title("Evolución de ventas")
ax.set_xlabel("Trimestre")
ax.set_ylabel("Miles de unidades")
ax.legend()
ax.grid(True)
# Mostrar figura
plt.show()
```



Esta estructura permite un control más fino sobre cada componente, en comparación con la interfaz procedural (`plt.plot()`, `plt.title()`...).

Comandos comunes de personalización

Antes de explorar más opciones de estilo y formato, es útil conocer una serie de métodos clave que permiten ajustar los gráficos generados:

Método	Descripción
ax.set_title()	Agrega un título a los ejes
ax.set_xlabel()	Asigna etiqueta al eje X
ax.set_ylabel()	Asigna etiqueta al eje Y
ax.grid(True)	Activa la cuadrícula
ax.legend()	Muestra la leyenda si se ha definido un label
fig.set_size_inches()	Ajusta el tamaño general del gráfico
plt.tight_layout()	Optimiza la distribución de elementos en la figura

Estos comandos pueden combinarse libremente según las necesidades del análisis o de presentación de resultados.

Buenas prácticas al construir gráficos

Antes de visualizar datos con Matplotlib, es esencial considerar una serie de prácticas que optimicen la legibilidad, precisión y utilidad del gráfico final.

Estas buenas prácticas son especialmente relevantes en contextos productivos donde los resultados deben ser compartidos con otras áreas.



Control total

Utiliza `fig, ax = plt.subplots()` en lugar de llamadas sueltas con `plt.*` para tener control total sobre la figura y los ejes.



Etiquetas claras

Define siempre etiquetas claras y títulos que contextualicen el gráfico.



Leyendas y colores

Si se muestran varias series de datos, incluye una leyenda y personaliza colores para diferenciarlas.



Optimización de layout

Aplica `tight_layout()` antes de mostrar el gráfico para evitar que se superpongan los textos.



Tamaño adecuado

Ajusta `figsize` según el medio donde se presentará (pantalla, informe, presentación).

Errores comunes

Pese a su flexibilidad, Matplotlib puede generar gráficos difíciles de interpretar si se omiten componentes claves o se emplean de manera incorrecta. A continuación, algunos errores frecuentes:

No cerrar figuras

Crear múltiples gráficos sin usar `plt.close()`, lo cual puede saturar la memoria.

Omitir etiquetas

Usar `plt.plot()` sin etiquetas, lo que dificulta la interpretación para otras personas.

Confundir terminología

Confundir axes con axis: los axes son áreas de dibujo, mientras que los axis son líneas de coordenadas.

Sin leyendas

No utilizar `legend()` cuando se comparan varias curvas, lo cual resta claridad al análisis.

Omitir cuadrícula

Omitir `grid()` cuando el gráfico lo requiere, dificultando la lectura numérica.

Dominio de la estructura de Matplotlib

Dominar la estructura interna de Matplotlib permite construir visualizaciones adaptadas a múltiples contextos técnicos. Un gráfico bien construido no solo muestra datos: cuenta una historia con evidencia visual clara y confiable.

Este conocimiento será clave en las siguientes sesiones, donde abordaremos cómo agregar estilos, anotar eventos, y componer gráficos complejos con múltiples dimensiones.



Actividad guiada: Construyendo visualizaciones claras desde cero



Ejercicio

Explorando patrones de uso con visualizaciones en Seaborn

Objetivo de la actividad

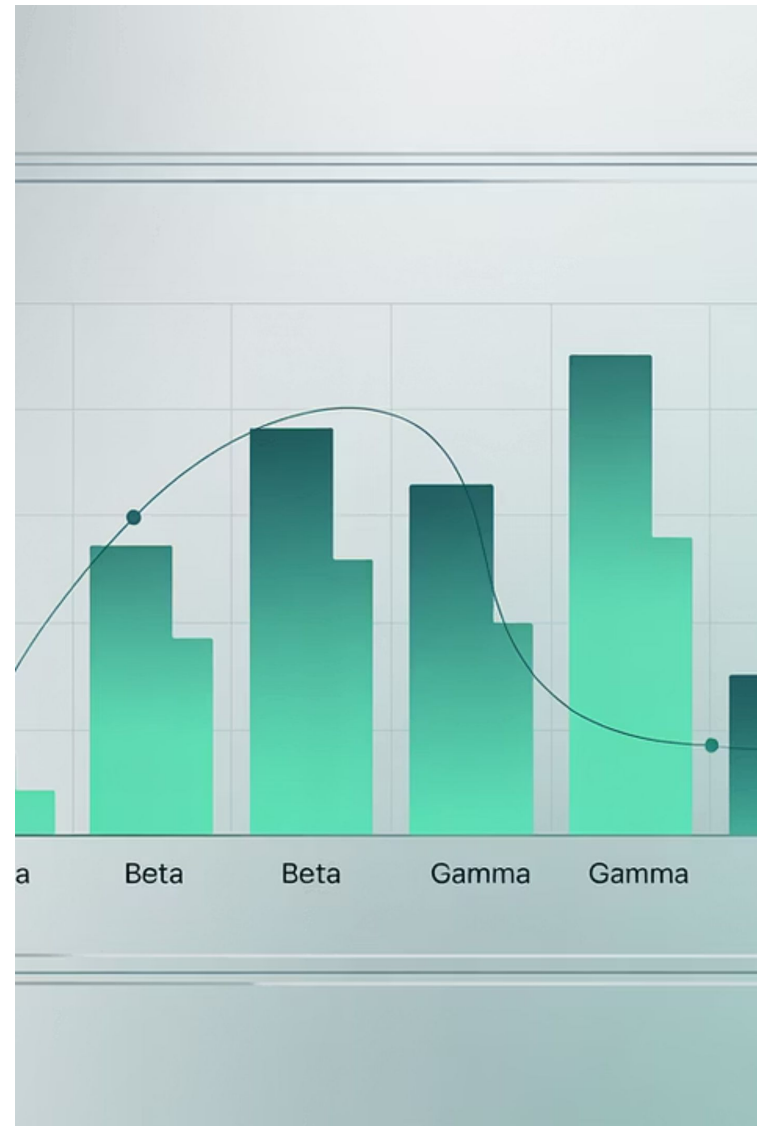
Aplicar los conceptos aprendidos sobre Matplotlib para construir una visualización efectiva y clara a partir de datos categóricos y numéricos, considerando los componentes fundamentales del gráfico.

Situación problema

Trabajas en una empresa de análisis de ventas y necesitas presentar un informe visual para la gerencia. Cuentas con datos sobre las ventas mensuales de cuatro productos distintos durante un trimestre. El equipo directivo requiere una gráfica clara que compare el rendimiento de estos productos de manera comprensible para la toma de decisiones.

¿Cómo construir un gráfico con Matplotlib que muestre comparativamente las ventas, incluyendo todos los elementos gráficos necesarios para facilitar su comprensión?





Conjunto de datos a utilizar

```
import matplotlib.pyplot as plt
meses = ["Enero", "Febrero", "Marzo"]
producto_A = [120, 135, 150]
producto_B = [80, 95, 100]
producto_C = [150, 160, 170]
producto_D = [100, 110, 130]
```

Ejercicio

Instrucciones

Paso 1: Carga las bibliotecas necesarias

```
import matplotlib.pyplot as plt
import pandas as pd
```

Paso 2: Crea los datos a visualizar

Simularemos una serie temporal con los datos proporcionados por tu equipo.

Ejecuta este código:

```
semanas = ["Semana 1", "Semana 2", "Semana 3", "Semana 4"]
entregas_zona_norte = [120, 135, 150, 160]
entregas_zona_sur = [100, 110, 95, 130]
```



Ejercicio

Instrucciones

Paso 3: Genera la visualización principal

Utiliza Matplotlib para graficar ambas series en una misma figura. Personaliza cada elemento gráfico:

```
fig, ax = plt.subplots()
ax.plot(semanas, entregas_zona_norte, label="Zona Norte",
marker='o')
ax.plot(semanas, entregas_zona_sur, label="Zona Sur",
marker='s')
ax.set_title("Entregas semanales por zona")
ax.set_xlabel("Semana")
ax.set_ylabel("Cantidad de entregas")
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()
```

Ejercicio

Instrucciones

Paso 4: Guarda el gráfico como imagen

Simula que necesitas enviar esta visualización por correo electrónico:

```
fig.savefig("entregas_semanales.png")
```



Resumen

-  Aprendimos qué es Matplotlib y por qué es una herramienta fundamental para la visualización de datos en Python.
-  Vimos cómo importar correctamente la librería y trabajar con su interfaz más común: pyplot.
-  Exploramos los elementos que componen un gráfico bien estructurado: figuras, ejes, títulos, etiquetas, leyendas y cuadrículas.
-  Conocimos buenas prácticas para asegurar gráficos legibles, reutilizables y estéticamente consistentes.
-  Identificamos errores frecuentes que pueden afectar la claridad o funcionalidad de las visualizaciones.



Próxima sesión...

En la próxima sesión profundizaremos en el uso práctico de Matplotlib, aprendiendo a crear gráficos desde cero con funciones de pyplot, comprenderemos el modelo de orientación a objetos para personalizar con mayor precisión nuestras visualizaciones, y exploraremos cómo diseñar figuras con múltiples subgráficos. Estas herramientas nos permitirán desarrollar visualizaciones más profesionales, reutilizables y adaptadas a distintos tipos de análisis, fortaleciendo nuestra capacidad para comunicar información en contextos productivos.